

Applied Cloud Computing

Assignment 1 : Part 2: Amazon ELB (Elastic Load Balancer) (updated Sept. 2025)

In this assignment, you will create two EC2 instances from an AMI template running http server and an ELB in front of the two EC2 instances to redirect http requests. Here are the steps:

1. Create a new EC2 instance, let's call it instance **A**.(refer to Part 1's steps, no need to install httpd nor create index.html file. Just create a new instance with HTTP traffic enabled. Do **not** reuse Part 1's instance directly).

2. Log on instance **A** using ssh and run the following commands

```
$ sudo yum update
```

```
$ sudo yum install python3-pip
```

```
$ sudo pip3 install flask
```

```
$ mkdir cloudcompute
```

```
$ vi cloudcompute/server.py
```

(with the following content and save (you can copy and paste the server.py file under the cloudcompute directory)

```
#!/usr/bin/python3
from flask import Flask, request
import socket
app = Flask(__name__)
@app.route('/', methods=['GET'])
def index():
    return "Hello world! from -- {}".format(socket.gethostname())
if __name__ == '__main__':
    app.run(host="0.0.0.0", port=80)
```

While editing the server.py file, make sure you follow Python's indentation rules.

```
$ sudo vi /etc/systemd/system/cloudcompute.service
```

(with the following content and save (you can copy the cloudcompute.service file to the /etc/systemd/system/ directory)

```
[Unit]
```

```
Description=cloudcompute web application
```

```
After=network.target
```

```
[Service]
```

```
User=ec2-user
```

```
WorkingDirectory=/home/ec2-user/cloudcompute
ExecStart=/usr/bin/sudo /usr/bin/python3 server.py
Restart=always
RestartSec=10s
[Install]
WantedBy=multi-user.target
```

Note: If your directory for the server.py is different than example, make sure you modify the path properly.

\$ sudo systemctl daemon-reload

\$ sudo systemctl start cloudcompute

\$ sudo systemctl status cloudcompute

(Verify the cloudcompute service is running. If you hit any errors, you need to check all previous steps to make sure they are 100% correct). Use Control-C to terminate the 'sudo systemctl status cloudcompute' process.

- Now use your local web browser and access your EC2 instance **A**'s web server by typing `http://your_ec2_instance_A_name`. In your local browser, you should see something like:

```
Hello world! from -- ip-172-31-16-109.us-west-1.compute.internal
```

- Explore the **\$ sudo systemctl status** command more yourself.

3. Create an AMI out of your EC2 instance **A**:

- Read detail instruction from here first: <https://docs.aws.amazon.com/toolkit-for-visual-studio/latest/user-guide/tkv-create-ami-from-instance.html>. Create your own AMI image from your instance **A**.
 - In EC2->Instance, select the instance you want to base on, right click -> Image and templates -> create images, fill in a name (ex: **CloudCompute_AMI_HW1**) then click 'Create image'.
- You can check the new image from EC2 **Images** -> **AMIs** (by default with "owned by me" filter. It may take a minute or so for it to become ready)
- After the new AMI image "**CloudCompute_AMI_HW1**" showed up correctly, now you can stop the instance **A**.

4. Create a new EC2 instance, based on the AMI image you just created.

- Select the "**CloudCompute_AMI_HW1**" AMI template, launch a new instance through Images -> AMIs -> Launch instance from AMI. Go through the similar step, give a proper name(ex: *HW1Part2Instance_1*), select the key pair, allow HTTP traffic, then click 'Launch Instance'.
- After a minute or so, you should see the new *HW1Part2Instance_1* running from the Instance View tab. (if not, refresh the Instance view)

- Use a ssh an login into *HW1Part2Instance_1* and start the cloudcompute.service via **"sudo systemctl start cloudcompute "**.
- In your local browser, open a new tab, connect to the instance *HW1Part2Instance_1*'s web server . Verify the web server is working correctly on instance *HW1Part2Instance_1*. In your local browser tab

5. Create a second EC2 instance based on the AMI you created in previous step by selecting the AMI image you just created in **Images -> AMIs**, give the proper name (ex:

HW1Part2Instance_2) , allow the HTTP traffic, then select **"Launch instance from AMI"**.

- After launch, Log on instance *HW1Part2Instance_2* using another ssh (with the proper keypair).
- After connect to the instance *HW1Part2Instance_2*, launch the cloudcompute.service (**\$ sudo systemctl start cloudcompute**)
- In your local browser, open another tab, connect to the instance *HW1Part2Instance_2*'s web server . Verify the web server is working correctly on instance *HW1Part2Instance_2*. In your local browser tab, you should see something like:
Hello world! from -- ip-172-31-22-55.us-west-1.compute.internal

Note: This line is similar with the instance *HW1Part2Instance_1*, but the IP address is **definitely different**.

6. Create an ELB(Elastic Load Balancer), registering the two EC2 instance *HW1Part2Instance_1* and *HW1Part2Instance_2* into ELB's target group. Read through the document about creating Application ELB.

- <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/application-load-balancer-getting-started.html>
- First you need to create target group via "Load Balancing" -> "Target Groups" -> create target group. target type -> select 'Instance'. Specify group name (ex: GroupHW1Part2). In next page, select the two instances (*HW1Part2Instance_1* and *HW1Part2Instance_2*), then create the target group GroupHW1Part2.
- Then we create load balancer via select "Application Load Balancer". Give a name (ex: ELBHW1Part2),
- In "Availability Zones and subnets", select at least two zones.
- In "Security Group", you need to add at least one more security group together with the 'default'. For example, both 'launch-wizard-1' and 'default' are selected.
- In 'Listeners and routing', ..In the 'Target group', make sure you select the 'target group'(GroupHW1Part2) you just created.
- In the last step, after you click "Create load balancer", it will take up to few minutes to have this balancer alive. Make sure you refresh then check the ELB status is "Active" before moving to next step.
- Verify the ELB works in local browser by typing http://your_ec2_ELB_DNS_name.
Note: If the ELB is working, you should see one of the A or B instance's "Hello world" message.

7. By refreshing your local browser tab in the `http://your_ec2_ELB_DNS_name` multiple times, observe the change in ipv4 address name shown in the page. You should see two different ipv4 addresses from the two instance **1** and **2** you put into the Load Balancer Target groups.

8. Now go to instance **1** command window, use editor like vi to modify the `server.py` file under `cloudcompute` directory: for example

\$ vi cloudcompute/server.py

and change the line `"def index()"` in `server.py` file into:

```
def index():  
    return "<h1>Hello world! </h1> from -- {}".format(socket.gethostname())
```

relaunch the `cloudcompute.service` by

\$ sudo systemctl restart cloudcompute.service

Now refresh your local browser `http://your_ec2_ELB_DNS_name`. Observe the differences.

9. We create the http server using python3 and flask, which can render dynamic web pages using jinja2 template engine. Follow the tutorial:

https://www.tutorialspoint.com/flask/flask_templates.htm) to create an `index.html` template file that follow flask's file directory structure. The `index.html` needs to return something like `"Hello World from EC2_Instance_Host_name"`. You need to modify the `server.py`'s `index()` function to pass EC2 instance hostname into the `index.html` template.

```
#!/usr/bin/python3  
from flask import Flask, request , render_template  
import socket  
app = Flask(__name__)  
@app.route('/', methods=['GET'])  
def index():  
    # put your code here  
if __name__ == '__main__':  
    app.run(host="0.0.0.0", port=80)
```

When done, restart the http server

\$ sudo systemctl restart cloudcompute

and verify your change works

HW1 Part 2, Deliverables

Document the creation of your own AMI that includes:
Q1, Screenshot of the AMI, EC2 console -> Images->AMIs-> (with Owned by me filter), with Details (AMI name, Status, Owner, Creation date etc.) (5 pts)

EC2

Dashboard

EC2 Global View

Events

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Images

AMIs

AMI Catalog

ami-0402e24b08eaea7aa

Image summary for ami-0402e24b08eaea7aa

AMI ID

ami-0402e24b08eaea7aa

AMI name

CloudCompute_AMI_HW1

Root device name

/dev/xvda

Boot mode

uefi-preferred

Description

Last launched time

Sun Sep 28 2025 12:41:45 GMT-0700 (Pacific Daylight Time)

Source AMI ID

ami-00142eb1747a493d9

Image type

machine

Owner account ID

293578648063

Status

Available

State reason

Product codes

Block devices

/dev/xvda=snap-083c4df1afe53820d:8:true:gp3

Source AMI Region

us-west-1

Platform details

Linux/UNIX

Architecture

x86_64

Source

293578648063/CloudCompute_AMI_HW1

Creation date

2025-09-28T19:33:36.000Z

RAM disk ID

Deregistration protection

Disabled

Root device type

EBS

Usage operation

RunInstances

Virtualization type

hvm

Kernel ID

Deprecation time

Allowed image

EC2 Image Builder

Actions

Launch instance from AMI

Document the creation of EC2 instance B that includes:
Q2, Screenshot of the EC2 instance description which shows instance state, Instance type, Availability Zone, Security Groups, Public DNS(ipv4). (5 pts)

Instance summary for i-0b8a37974176c06c9 (HW1Part2Instance_1)

Updated less than a minute ago

Instance ID

i-0b8a37974176c06c9

IPv6 address

Hostname type

IP name: ip-172-31-24-134.us-west-1.compute.internal

Answer private resource DNS name

IPv4 (A)

Auto-assigned IP address

54.219.232.121 [Public IP]

IAM Role

IMDSv2

Required

Operator

Public IPv4 address

54.219.232.121 | open address

Instance state

Running

Private IP DNS name (IPv4 only)

ip-172-31-24-134.us-west-1.compute.internal

Instance type

t3.micro

VPC ID

vpc-091ffc5ef10a67c8f

Subnet ID

subnet-05dca873c49f84a59

Instance ARN

arn:aws:ec2:us-west-1:293578648063:instance/i-0b8a37974176c06c9

Private IPv4 addresses

172.31.24.134

Public DNS

ec2-54-219-232-121.us-west-1.compute.amazonaws.com | open address

Elastic IP addresses

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Auto Scaling Group name

Managed

false

Details	Status and alarms	Monitoring	Security	Networking	Storage	Tags
VPC ID vpc-091ffc5ef10a67c8f			Subnet ID subnet-05dca873c49f84a59		Availability zone us-west-1a	
Availability zone ID usw1-az1			Outpost ID -			
Public IPv4 address 54.219.232.121 open address			Private IPv4 addresses 172.31.24.134		IPv6 addresses -	
Secondary private IPv4 addresses -			Carrier IP addresses (ephemeral) -			
▼ Hostname and DNS Info			Public DNS ec2-54-219-232-121.us-west-1.compute.amazonaws.com open address		Private IP DNS name (IPv4 only) ip-172-31-24-134.us-west-1.compute.internal	
Dualstack - IP based name: A and AAAA record -			IPv6-only - IP based name: AAAA record only -		IPv4-only IP based name: A record only ec2-54-219-232-121.us-west-1.compute.amazonaws.com	
Private hostname type IP name: ip-172-31-24-134.us-west-1.compute.internal			Use RBN as guest OS hostname Disabled		Public hostname type public-ipv4-dns-name	
Answer RBN DNS hostname IPv6 -			Answer private resource DNS name IPv4 (A)		Answer RBN DNS hostname IPv4 Enabled	
Details	Status and alarms	Monitoring	Security	Networking	Storage	Tags
▼ Security details			IAM Role -		Owner ID 293578648063	
Security groups sg-06456eafdc511f703 (launch-wizard-6)					Launch time Mon Sep 29 2025 11:41:57 GMT-0700 (Pacific Daylight Time)	

Q3, Screenshot of verifying the running of http server on the console. (5 pts)

```
[ec2-user@ip-172-31-18-200 ~]$ sudo systemctl status cloudcompute
● cloudcompute.service - cloudcompute web application
   Loaded: loaded (/etc/systemd/system/cloudcompute.service; enabled; preset: disabled)
   Active: active (running) since Mon 2025-09-29 18:42:07 UTC; 5min ago
   Main PID: 1427 (sudo)
     Tasks: 0 (limit: 1057)
    Memory: 3.6M
       CPU: 45ms
    CGroup: /system.slice/cloudcompute.service
            └─ 1427 /usr/bin/sudo /usr/bin/python3 server.py

Sep 29 18:42:07 ip-172-31-18-200.us-west-1.compute.internal systemd[1]: Started cloudcompute.service - cloudcompute web application.
Sep 29 18:42:07 ip-172-31-18-200.us-west-1.compute.internal sudo[1427]: ec2-user : PWD=/home/ec2-user/cloudcompute ; USER=root ; COMMAND=/usr/bin/
```

Q4, Screenshot of verifying the running of http server on the browser. (5 pts)



Document the creation of ELB that includes:

Q5, Screenshot of the ELB description. (5 pts)

ELBHW1Part2 Actions

▼ Details

Load balancer type Application	Status Active	VPC vpc-091ffc5ef10a67c8f	Load balancer IP address type IPv4
Scheme Internet-facing	Hosted zone Z368ELLRRE2KJO	Availability Zones subnet-05dca873c49f84a59 us-west-1a (usw1-az1) subnet-0e7991bcf1aba410c us-west-1c (usw1-az3)	Date created September 29, 2025, 01:59 (UTC-07:00)

Load balancer ARN
[arn:aws:elasticloadbalancing:us-west-1:293578648063:loadbalancer/app/ELBHW1Part2/eb3a6e307f65f1b1](#)

DNS name [Info](#)
[ELBHW1Part2-1054883007.us-west-1.elb.amazonaws.com](#) (A Record)

Listeners and rules | Network mapping | Resource map | Security | Monitoring | Integrations | Attributes | Capacity | Tags

Listeners and rules (1) [Info](#) Manage rules Manage listener Add listener

A listener checks for connection requests on its configured protocol and port. Traffic received by the listener is routed according to the default action and any additional rules.

☐	Protocol:Port	Default action	Rules	ARN	Security policy	Default SSL
☐	HTTP:80	Forward to target group GroupHW1Part2: 1 (100%) Target group stickiness: Off	1 rule	ARN	Not applicable	Not applicab

Q6, Screenshot of the “Load balancer -> Security Groups” (5 pts)

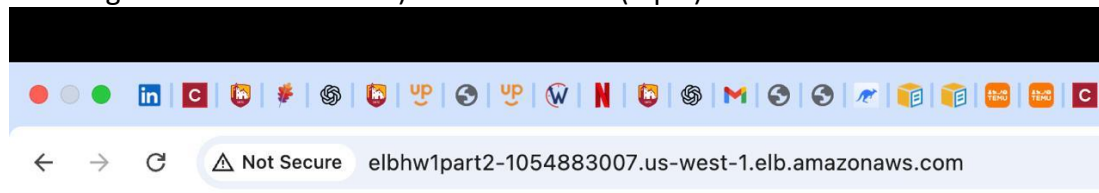
Listeners and rules | Network mapping | Resource map | **Security** | Monitoring | Integrations | Attributes | Capacity | Tags

Security groups (2) Edit

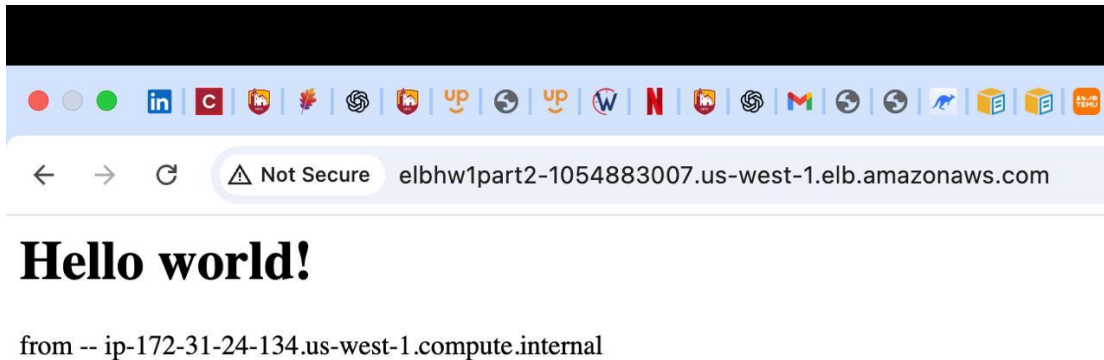
A security group is a set of firewall rules that control the traffic to your load balancer.

Security Group ID	Name	Description
sg-06456eafdc511f703	launch-wizard-6	launch-wizard-6 created 2025-09-28T19:38:40.417Z
sg-0dd888731610df344	default	default VPC security group

Q7, Two screenshots of verifying ELB works (show web contents from instance A & instance B through the same ELB’s URL) in the browser. (5 pts)



Hello world! from -- ip-172-31-18-200.us-west-1.compute.internal



Coding - dynamic http server using flask template that includes:

Q8, Content of your new index.html and your new server.py. (5 pts)

```
[ec2-user@ip-172-31-18-200 templates]$ cat index.html
<!doctype html>
<html lang="en">
  <head><meta charset="utf-8"><title>CloudCompute</title></head>
  <body>
    <h1>Hello World</h1>
    <p>from {{ hostname }}</p>
  </body>
</html>
[ec2-user@ip-172-31-18-200 templates]$ █

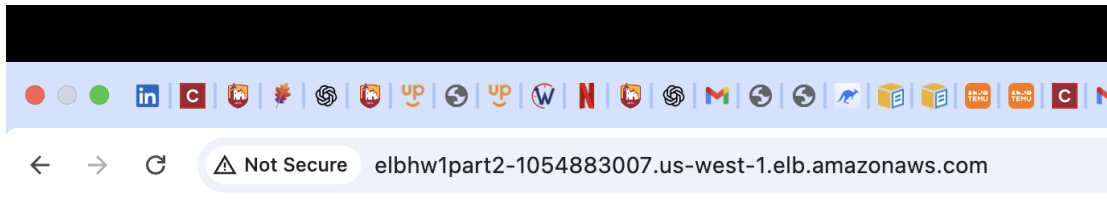
[ec2-user@ip-172-31-18-200 cloudcompute]$ cat server.py
#!/usr/bin/python3
from flask import Flask, request, render_template
import socket

app = Flask(__name__)

@app.route('/', methods=['GET'])
def index():
    # pass EC2 instance hostname into the template
    return render_template('index.html', hostname=socket.gethostname())

if __name__ == '__main__':
    app.run(host="0.0.0.0", port=80)
[ec2-user@ip-172-31-18-200 cloudcompute]$ █
```

Q9, Screenshot of the ELB browser page which shows the modified flask template (5 pts)



Hello World

from ip-172-31-18-200.us-west-1.compute.internal

Q10, Make sure you are in the `cloudcompute` directory you created then generate a “file tree” by installing “tree” utility with following commands. Submit the screenshot of your “file tree”. (5 pts)

```
$ sudo yum install tree
```

```
$ tree
```

```
[[ec2-user@ip-172-31-18-200 cloudcompute]$ tree
```

```
.
├── server.py
└── templates
    └── index.html
```

```
1 directory, 2 files
```

```
[ec2-user@ip-172-31-18-200 cloudcompute]$ █
```